SENAI

JSON, Arquivos e

Persistência Local

Aula 04 | Programação Back-end 2

Projeto: catálogo de produtos persistido em JSON

Ao final da aula

SENAI

Converter

Separar storage, catálogo persistente e orquestração CLI.

Persistir

Ler e gravar arquivos de forma assíncrona com fs/promises.

Organizar

Separar utilitário de I/O, serviço de produtos e interface CLI.

Capacidades e evidências

- Converter entre objetos JavaScript e texto JSON.
- Criar catalogoArquivo com ID incremental e escrita segura.
- Tratar arquivo inexistente sem ocultar outras falhas.
- Criar ProdutoArquivoService com ID incremental.
- Interpretar comandos usando process.argv.
- Comprovar que os dados permanecem entre execuções.

Roteiro da aula

JSON
Formato e conversão

Arquivos
Leitura e escrita

Service
Regra da aplicação

CLI
Comandos no terminal

Cada camada conhece apenas a responsabilidade necessária.

Mão na massa: Preparação

```
mkdir exemplos\aula04
cd exemplos\aula04
```

Mini-desafio 1: Objeto ou texto? (conversao.js)

```
const produto = {
  id: 1,
  nome: 'Teclado mecânico',
  preco: 249.9,
  disponivel: true,
};

const texto = JSON.stringify(produto, null, 2);
const reconstruido = JSON.parse(texto);

console.log(typeof produto, typeof texto, typeof reconstruido);
console.log(texto);
```

Mini-desafio 2: Perdas na serialização (conversao.js)

```
// undefined é completamente ignorado pelo JSON
console.log(JSON.stringify({ nome: 'Sensor', observacao:
undefined }));

// Objetos circulares explodem o Node.js
const circular = { nome: 'Produto' };
circular.referencia = circular;
console.log(JSON.stringify(circular));
```

Mini-desafio 3: Os dados em UTF-8 (data/produtos.json)

```
mkdir data

// data/produtos.json
[
  {
  "id": 1,
    "nome": "Teclado",
    "preco": 120
  }
]
```

Mini-desafio 3: Leitura em UTF-8 (leitura.js)

```
import { readFile } from 'node:fs/promises';
import { resolve } from 'node:path';

const caminho = resolve(import.meta.dirname, 'data',
'produtos.json');
const texto = await readFile(caminho, 'utf8');
const dados = JSON.parse(texto);

if (!Array.isArray(dados)) {
  throw new TypeError('O catálogo JSON deve conter um array.');
}

console.log(dados);
```

Mini-desafio 4: Tratar somente arquivo ausente (leitura.js)

```
async function lerJson(caminho) {
  try {
    const texto = await readFile(caminho, 'utf8');
    const dados = JSON.parse(texto);
    if (!Array.isArray(dados)) throw new TypeError('Deve conter um
array.');
  return dados;
  } catch (erro) {
    if (erro.code === 'ENOENT') return []; // Arquivo sumiu? Retorna
vazio!
    throw erro; // Outros erros explodem normalmente
  }
}
```

Mini-desafio 5: Gravar e reler (escrita.js)

```
import { readFile, writeFile } from 'node:fs/promises';
import { resolve } from 'node:path';

const caminho = resolve(import.meta.dirname, 'data', 'saida.json');
const produtos = [{ id: 1, nome: 'Mouse', preco: 80 }];
const texto = `${JSON.stringify(produtos, null, 2)}\n`;

await writeFile(caminho, texto, 'utf8');
console.log(JSON.parse(await readFile(caminho, 'utf8')));
```

Mini-desafio 6: Substituir atômico (atomica.js)

Preparar o temporário

```
import { readFile, rename, unlink, writeFile } from
'node:fs/promises';

const pasta = new URL('./data/', import.meta.url);
const destino = new URL('produtos-atomico.json', pasta);
const temporario = new URL(`produtos-${process.pid}.tmp`,
pasta);
const falharAntesDoRename = process.argv.includes('--falhar');

const dados = [ { id: 1, nome: 'Teclado' }, { id: 2, nome: 'Mouse' } ];

try {
  const texto = `${JSON.stringify(dados, null, 2)}\n`;
  await writeFile(temporario, texto, 'utf8'); // Grava no .tmp

  if (falharAntesDoRename) throw new Error('Falha simulada
antes do rename.');
```

Substituir ou limpar

```
  await rename(temporario, destino); // Renomeia só
se deu certo
  console.log(JSON.parse(await readFile(destino, 'utf8')));
} catch (erro) {
  await unlink(temporario).catch((err) => { if (err.code
!== 'ENOENT') throw err; });
  console.error(erro.message);
  process.exitCode = 1;
}
```

SENAI

3. Evolução hands-on da api-produtos

Evolua a API cumulativa da Aula 03, arquivo por arquivo

Passo 1: volte à API e remova o mock

```
cd ..\api-produtos
del data\catalogo.js
```

Passo 1: crie data/produtos.json

```
[
  {
    "id": 1,
    "nome": "Teclado",
    "preco": 120,
    "estoque": 4,
    "categoria": "Periféricos"
  }
]
```

Passo 2: crie a camada Storage

```
mkdir storage
```

Passo 2: storage/arquivoJson.js | 1 de 3

```
import { mkdir, readFile, rename, rm, writeFile } from
'node:fs/promises';
import { randomUUID } from 'node:crypto';
import { basename, dirname, join } from 'node:path';
export async function lerJson(caminho) {
  let texto;
  try {
    texto = await readFile(caminho, 'utf8');
  } catch (erro) {
    if (erro.code === 'ENOENT') return [];
    throw erro;
  }
```

Passo 2: storage/arquivoJson.js | 2 de 3

```
  try {
    const dados = JSON.parse(texto);
    if (!Array.isArray(dados)) {
      throw new TypeError('O catálogo JSON deve conter
um array.');
    }
    return dados;
  } catch (erro) {
    if (erro instanceof SyntaxError) {
      throw new SyntaxError(`JSON inválido em
${basename(caminho)}.`);
    }
    throw erro;
  }
}
```

Passo 2: storage/arquivoJson.js | 3 de 3

```
export async function gravarJsonComSeguranca(caminho, dados)
{
  if (!Array.isArray(dados)) {
    throw new TypeError('Os dados gravados devem formar um
array.');
  }
  const diretorio = dirname(caminho);
  await mkdir(diretorio, { recursive: true });
  const temporario = join(diretorio,
`.${basename(caminho)}.${randomUUID()}.tmp`);
  const texto = `${JSON.stringify(dados, null, 2)}\n`;
  try {
    await writeFile(temporario, texto, 'utf8');
    await rename(temporario, caminho);
  } catch (erro) {
    await rm(temporario, { force: true }).catch(() =>
undefined);
    throw erro;
  }
}
```

Passo 3: crie o catálogo persistente

catalogo/catalogoArquivo.js

Passo 3: catalogoArquivo.js | 1 de 2

```
import Produto from '../models/Produto.js';
import { gravarJsonComSeguranca, lerJson } from
'../storage/arquivoJson.js';
export function criarCatalogoArquivo(caminho) {
  async function listar() {
    const dados = await lerJson(caminho);
    return dados.map((produto) => new Produto(produto));
  }
  async function buscarPorId(id) {
    const produto = (await listar()).find((item) =>
item.id === id);
    if (!produto) throw new Error(`Produto ${id} não
encontrado.`);
    return produto;
  }
```

Passo 3: catalogoArquivo.js | 2 de 2

```
  async function listarCategorias() {
    const categorias = (await listar()).map(({ categoria
}) => categoria);
    return [...new Set(categorias)];
  }
  async function criar(dados) {
    const produtos = await listar();
    const proximoId = Math.max(0, ...produtos.map(({ id })
=> id)) + 1;
    const produto = new Produto({ id: proximoId, ...dados
});
    const atualizados = [...produtos, produto];
    await gravarJsonComSeguranca(caminho, atualizados);
    return produto;
  }
  return { listar, buscarPorId, listarCategorias, criar };
}
```

Passo 3: remova as consultas antigas

```
del catalogo\consultas.js
```

Passo 4: substitua todo o app.js

app.js

Passo 4: app.js | 1 de 4

```
import { resolve } from 'node:path';
import { criarCatalogoArquivo } from
'./catalogo/catalogoArquivo.js';
import { carregarAmbiente, exibirDiagnostico } from
'./config/ambiente.js';
import { formatarMoeda } from
'./utils/formatarMoeda.js';
function resumirProduto(produto) {
  return {
    id: produto.id,
    nome: produto.nome,
    preco: produto.preco,
    precoFormatado: formatarMoeda(produto.preco),
    estoque: produto.estoque,
    categoria: produto.categoria,
```

Passo 4: app.js | 2 de 4

```
    valorEmEstoque: produto.calcularValorEmEstoque()
  };
}
async function executar() {
  try {
    const configuracao = carregarAmbiente(process.argv[2]);
    const comando = process.argv[3] || 'listar';
    const caminhoPadrao = resolve(import.meta.dirname,
'data/produtos.json');
    const caminhoCatalogo = process.env.CATALOGO_ARQUIVO ||
caminhoPadrao;
    const catalogo = criarCatalogoArquivo(caminhoCatalogo);
    exibirDiagnostico(configuracao);
    if (comando === 'listar') {
      console.table((await
catalogo.listar()).map(resumirProduto));
```

Passo 4: app.js | 3 de 4

```
    } else if (comando === 'buscar') {
      const id = Number(process.argv[4]);
      if (!Number.isInteger(id)) throw new
Error('Informe um identificador inteiro.');
      console.log(resumirProduto(await
catalogo.buscarPorId(id)));
    } else if (comando === 'categorias') {
      console.log(await catalogo.listarCategorias());
    } else if (comando === 'criar') {
      const produto = await catalogo.criar({
        nome: process.argv[4],
        preco: Number(process.argv[5]),
        estoque: Number(process.argv[6] || '0'),
        categoria: process.argv[7] || 'Geral'
      });
```

Passo 4: app.js | 4 de 4

```
      console.log(resumirProduto(produto));
    } else {
      throw new Error('Use: listar | buscar <id> |
categorias | criar <nome> <preco> [estoque]
[categoria]');
    }
  } catch (erro) {
    console.error(erro.message);
    process.exitCode = 1;
  }
}
executar();
```

Passo 5: atualize os scripts

```
"scripts": {
    "start": "node app.js .env listar",
    "buscar": "node app.js .env buscar 1",
    "categorias": "node app.js .env categorias",
    "produto-ausente": "node app.js .env buscar 999",
    "id-invalido": "node app.js .env buscar abc",
    "diagnostico": "node app.js",
    "check": "node scripts/check.js"
  },
```

Passo 6: teste sucesso e persistência

```
npm run check
npm start
node app.js .env criar "Mouse ergonômico" 149.90 8
Periféricos
npm start
```

Passo 6: teste as falhas controladas

```
npm run produto-ausente
npm run id-invalido
node app.js .env comando-desconhecido
```

Persistência comprovada

```
criar → .tmp → rename → produtos.json

encerrar → npm start → Mouse ergonômico continua lá
```

Exercícios de consolidação

- Criar outro produto e confirmar o próximo ID.
- Reiniciar o processo e conferir a persistência.
- Renomear produtos.json e observar o retorno vazio.
- Malformar o JSON e confirmar o SyntaxError específico.
- Explicar a função do arquivo temporário e do rename.
- Verificar que nenhuma falha corrompe o catálogo válido.

Clean Code da aula

- Responsabilidade: storage, catálogo e app.js separados.
- Erro específico: somente ENOENT vira coleção vazia.
- Caminho previsível: import.meta.dirname ancora produtos.json.
- Validação central: Produto protege as regras intrínsecas.
- Nomes claros: lerJson, gravarJsonComSeguranca, listar e criar.

Erros comuns e diagnóstico

- JSON inválido virou []: não capture toda falha como ausência.
- Arquivo ausente virou erro: confira erro.code === ENOENT.
- Destino ficou parcial: grave o temporário antes do rename.
- Sobrou .tmp: confira a limpeza executada no catch.
- Buffer na leitura: confira o argumento utf8.
- Estrutura errada: valide Array.isArray depois do parse.

Critérios de avaliação

Persistência

Produtos permanecem em data/produtos.json entre execuções.

Robustez

Temporário, rename e limpeza protegem o JSON válido.

Arquitetura

Storage, catálogo e orquestrador têm papéis separados.

Evidências para concluir a aula

- JSON legível e indentado com dois espaços.
- lerJson e gravarJsonComSeguranca funcionando.
- catalogoArquivo criando Produto com ID incremental.
- app.js executando listar, buscar, categorias e criar.
- Produto ausente, ID inválido e comando desconhecido tratados.
- Código organizado em data, storage, catalogo, models e config.

Referências: JSON

JSON: formato e sintaxe

json.org/json-en.html

JSON.stringify

MDN: JSON.stringify

JSON.parse

MDN: JSON.parse

Referências: Node.js

SENAI

File system promises

nodejs.org/api/fs.html

Argumentos process.argv

nodejs.org/api/process.html

Utilitários de caminho

nodejs.org/api/path.html

fileURLToPath

nodejs.org/api/url.html